Adaptive Bitrate (ABR) Streaming

Table of Contents

- Creation of a server manifest file
- Options for LIVE ingest
- Pure LIVE
- Pure LIVE with archiving
- DVR with archiving
- Trick play for Live (DASH and HLS)
- Streams switching from Live (dynamic) to VOD (static)
- Alignment of sequence numbers from UTC
- Event mode
- Ingest Separate Events
- Ingest F4M (deprecated)

The streaming module can also act as a Publishing Point.

A Publishing Point is simply a URL that accepts input streams from one or more software/hardware encoders.

The encoder should follow interface 1 of the Interface 1 of the DASH-IF Live Media Ingest specification (i.e., CMAF ingest) to send the audio/video fragments to the web server. See the Supported Encoders section in the factsheet and LIVE Ingest for an overview.

> **Attention**
>
> Apache should be used for Live streaming (ingest and egress).

> **Note**
>
> The client manifest that Origin generates for a live stream will switch from being 'dynamic' (i.e., Live) to being 'static' in certain circumstances: Streams switching from Live (dynamic) to VOD (static).

Creation of a server manifest file

If you're publishing LIVE streams to the web server module, and no server manifest file is available then the default settings are used.

If you want to change any settings (e.g. adding on-the-fly encryption) you have to generate the server manifest file before starting the encoder.

In its simplest form the command for creating a LIVE server manifest file is:

```
#!/bin/bash

mp4split -o /var/www/live/channel1/channel1.isml
```

Alternatively, the LIVE server manifest can also be created if the web server module Live API is enabled as explained in Publishing Point API.

```
#!/bin/bash

mp4split -o http://api.example.com/live/channel1/channel1.isml
```

> **Note**
>
> The extension of the LIVE server manifest is **.isml**

Note the absence of any input files in the LIVE case. When the encoder pushes a live stream to the web server module for ingest, it is the *Unified Origin* that updates the LIVE server manifest file to include the stream information announced by the encoder.

Options available for both VOD also apply to LIVE and are described in Options for VOD and LIVE streaming.

The Publishing Point API documentation outlines the available commands with which you can control a publishing point.

Output fMP4 HLS

New in version 1.8.3.

You can change the HLS output format so that it uses fMP4 instead of Transport Streams by adding the `--hls.fmp4` command-line parameter when creating the Live server manifest.

For example:

```
#!/bin/bash

mp4split -o /var/www/live/channel1/channel1.isml --hls.fmp4
```

> **Note**
>
> To stream HEVC and/or HDR content over HLS according to Apple's specification, using fMP4 is a requirement.

File permissions

The web server module needs permission to read and write the DocumentRoot. Depending on your setup you may have to add read/write permissions to the directory and files.

On Linux you change the permissions of the file to allow read and write for all by using `chmod`:

```
#!/bin/bash

chmod ua+w /var/www/live
```

On Windows:

```
IIS: Select wwwroot properties
=> select Security
=> select Internet Guest Account
=> Tick Allow 'Read' and 'Write'.
```

Encoder URL

The URL to be passed to the encoder has the following format:

```
http://<server>/<pubpoint>/Streams(<identifier>)
```

Encoders append the `/Streams(<identifier>)` section themselves as specified in Interface 1 of the DASH-IF Live Media Ingest specification.

This means that you can simply use the URL ending in .isml with your encoder. For example:

```
http://live.example.com/channel1/channel1.isml
```

With FFmpeg the `/Streams(identifier)` should be added to the URL:

```
http://live.example.com/channel1/channel1.isml/Streams(ID)
```

Note the trailing `/Streams(ID)` where `ID` is a placeholder for your own identifier, which could be 'channel1' etc.

Options for LIVE ingest

Configuring the LIVE archive

> **Attention**
>
> It is highly recommended to explicitly configure all Live archive related options (`archiving`, `archive_length`, and `archive_segment_length`), and not rely on their defaults (as these defaults will lead to unexpected results).
>
> Also, changing any of these options requires a reset of the publishing point. Changing them on a publishing point that is in use, or that has been used, will break your publishing point and therefore break your stream.

--archiving

Always enable this option (by explicitly setting it to 1). Defaults to 0, which will still result in only the last two archive segments being kept on disk (if this is exactly what you want, it is better to set this option to 1 and simply configure the other Live archive related options according to your preference).

--archive_cmaf

New in version 1.11.15.

EXPERIMENTAL Ingested fmp4 is stored as CMAF media tracks, rather than ismv. Traditionally, *Unified Origin* stores the ingested multiplexed media as-is in archive segment files and maintains a fragment index in a database file. When `archive_cmaf` is enabled, the stream is demultiplexed into separate tracks. Each track is written to a separate file which includes a SegmentIndexBox (`sidx`). Ingested fragments are routed and appended to the corresponding archive segment and entries are gradually fill up the segment index.

A so-called Storage MPD provides high-level description of the ingested media contained in individual CMAF archive segments. Because neither ingest nor playout rely on the database index, this potentially provides better scalability.

--archive_length

The length of archive to be kept (in seconds). Archive segments beyond this range (measured from the live edge) will be automatically purged to free up disk storage. Note that the Origin will always have one (partial) 'open' live archive segment that it is writing to, which will not be purged.

> **Attention**
>
> The archive_length **must** be longer than the --dvr_window_length.

--archive_segment_length

If specified, the live presentation is archived in segments of the specified length (default to 0 seconds, meaning no segmentation takes place).

--dvr_window_length

Length of DVR moving window (default 30 seconds). Set to '0' to enable 'event mode', which will always return complete archive. Beware that this will increase load on Origin super linearly as the archive grows larger. Expect performance impact from both calculating the timeline for each client manifest request as well as an increase in distinct media segment requests because an ever growing window offers ever more media for viewers to scrub through.

> **Attention**
>
> The dvr_window_length **must** be shorter than the --archive_length.

--database_path

Specifies the location of the `.db3` file, so both ingest and playout share the same database. This option is ignored when `--archive_cmaf=true`. The path to the `.db3` file must be absolute and is specified like this:

```
#!/bin/bash

mp4split --database_path=/var/www/live/channel00/channel00.db3 -o test.isml
```

> **Note**
>
> Changing this option requires a reset of the publishing point.

--restart_on_encoder_reconnect

When this option is enabled, an encoder can reconnect and keep posting to a stream even after that stream was 'stopped' by an *end of stream* (EOS) signal (provided the stream layout is the same and the next timestamps are higher).

This is crucial when the encoder falls over and 'accidentally' sends the EOS signal. If the `--restart_on_encoder_reconnect` option is not enabled in such circumstances, the encoder will not be able to continue posting the livestream without a reset of the publishing point. Therefore, enabling this option is highly recommended.

The encoder needs to be configured to use Must Fix: use of UTC timestamps as the time it uses, please refer to the encoder manual on how to configure this.

--storage_mpd_url

New in version 1.12.7.

Specifies the location of the storage MPD when ingested media is archived as CMAF. By default, this Storage MPD is named after the server manifest, but with `.isml` replaced by `.smpd` (or just `.mpd` in versions prior ro 1.13.3). It is possible to set this to a different name using `--storage_mpd_url`. This is useful when pointing a distinct (egress-only) server manifest configuration at a shared CMAF archive.

An absolute storage MPD URL can be local (`file://`) as well as remote (**experimental**, `http://` or `https://`). A relative path will be resolved against file system location of the server manifest.

--time_shift

The time shift offset (in seconds). Defaults to 0.

> **Note**
>
> Because the use of `time_shift` only affects which segments are announced in the client manifest and the correct behavior for DASH clients is to calculate which segments are available based on the 'current' time, simply using `time_shift` may not result in the expected behavior (i.e., a DASH client may request the segments closest to the live edge irrespective of the `time_shift` offset).
>
> To ensure correct behavior by DASH clients, offset the `MPD@availabilityStartTime` equal to the `time_shift`:
>
> ```
> --time_shift=3600 --mpd.availability_start_time=3600
> ```
>
> This not only results in the latest segment announced in the client manifest being 3600 seconds behind the actual live edge, but also shifts the entire MPD timeline 3600 seconds into the future without changing the addressing of the actual segments. So when a DASH client calculates the latest media segment that is available in this scenario, it will now request content from about an hour ago.

Schematically

The options are related as depicted by the following picture:

```
                                        dvr_window_length
                                                |
                |-------------------------------*------|
         archive_length                                 ^
                                                   live point
                             < time_shift |
                                          ^
                                 (new 'live' point)


1. each '-' is an archive segment, set by 'archive_segment_length'

2. 'archive_length' is used to set the total length of the archive

3. 'archiving' is used to turn the feature on or off (without archiving only two segments are kept
on disk)

4. 'time_shift' offsets the live point (and DVR window) backwards but within the 'archive_length'
```

Pure LIVE

The following commands create a LIVE server manifest file for presentations where only a very short archive of two segments of 60 seconds is stored on disk, and the DVR window available is 30 seconds:

```bash
#!/bin/bash

mp4split -o channel1/channel1.isml \
  --archiving=1 \
  --archive_length=120 \
  --archive_segment_length=60 \
  --dvr_window_length=30
```

Pure LIVE with archiving

If you wish to publish content to your catch-up service 'archiving' can be added. The webserver maintains a 'rolling buffer' storing content within the set `--archive_length` (86400s = 24hrs):

```bash
#!/bin/bash

mp4split -o channel2/channel2.isml \
  --archiving=1 \
  --archive_length=86400 \
  --archive_segment_length=3600 \
  --dvr_window_length=30
```

DVR with archiving

Here previous settings are combined: an archive is maintained and a specific length is set for the DVR window, allowing the viewer to rewind back in time. As the content is archived catch-up clips can be created from the archive.

```
#!/bin/bash

mp4split -o channel3/channel3.isml \
  --archiving=1 \
  --archive_length=3600 \
  --archive_segment_length=1200 \
  --dvr_window_length=600 \
  --restart_on_encoder_reconnect
```

See the Getting Started with Live section for a full example.

Trick play for Live (DASH and HLS)

For HLS, an I-frame Playlist is automatically generated when streaming HLS TS with a client manifest version of `4` or higher (see How to add trick play to HLS TS), or when streaming HLS fMP4. In uncommon case the live encoder is configured to sent multiple GOPs per segment, --fixed_gop (Live only) can be used for both HLS TS and HLS fMP4 to ensure the highest possible frequency of I-frames signaled in the I-frame Playlist.

For DASH, trick play can be enabled by specifying the `fixed_gop` option as explained in Without a sync-samples only or tiled thumbnails track (Live only).

Streams switching from Live (dynamic) to VOD (static)

The client manifest that *Unified Origin* generates for a live stream will switch from being 'dynamic' (i.e., Live) to being 'static' in two circumstances:

- When a live stream has ended and the encoder has send an End of Stream signal (more info: Overview of possible publishing point 'states')
- When the end time of a virtual subclip (more info: Virtual subclips) from a livestream goes from being in the future to being in the past

This behavior is as expected and according to spec (e.g., see section 4.6 'Provisioning of Live Content in On-Demand Mode' of the DASH-IF Interoperability Points).

Note

When Origin switches an MPD (i.e., a DASH client manifest) from dynamic to static, it removes the `availabilityStartTime`, `timeShiftBufferDepth` and `minimumUpdatePeriod` attributes from the MPD. It also adds a `presentationTimeOffset` attribute to offset the timeline of each track so that the URLs for all segments will remain the same (thereby increasing caching efficiency).

Alignment of sequence numbers from UTC

New in version 1.6.0.

Sequence numbers for media segments are derived from the (UTC) timestamps that correspond to the segments. This guarantees that two Origins use the same sequence numbers when generating a HLS Media Playlist or an MPD (if the latter references segments using `$Number$`, otherwise timestamps are used instead of sequence numbers).

A segment's sequence numbers is calculated as follows:

```
floor(timestamp of fragment (in seconds) / fragment_duration (in seconds))
```

Event mode

A live stream is considered an event when all content is archived on disk and nothing is purged. That is, the archive will be as long as the duration of the entire event.

In general, events have an infinite DVR window as well, so that it's always possible to scrub back to the beginning of the event.

```bash
#!/bin/bash

mp4split -o channel1.isml \
  --archiving=1 \
  --archive_length=0 \
  --dvr_window_length=0
```

Note

When you specify an infinite DVR window (`--dvr_window_length=0`), the HLS Media Playlist will contain specific signaling to indicate that the stream is an event: '#EXT-X-PLAYLIST-TYPE:EVENT'. See also Apple's HLS documentation on 'Event Playlist Construction'.

Ingest Separate Events

Publishing points can be re-used by adding an `EventID` to the publishing point url. This creates separate archives for each event while maintaining the same play-out URL. Typically this can be selected in the encoder UI. The URL is formatted as follows:

```
http(s)://<domain>/<path>/<ChannelName>.isml/Events(<EventID>)/Streams(<StreamID>)
```

Starting an encoding session with a specified EventID will add an extra line to a server manifest, referring to the EventID:

```
<meta name="event_id" content="2013-01-01-10_15_25">
```

Given the example above, the stream's Live archive and SQLite database would be stored within the following (automatically created) subdirectory of the publishing point:

```
2013-01-01-10_15_25/
```

Do note that a unique EventID must be used for each Live presentation that makes use of the same publishing point. The best way to achieve this is to use a stream's start date and time as its EventID.

Playout of streams with different EventIDs

When a publishing point is re-used with a new EventID, the server manifest will be associated with the new instead of the old event. Thus, from then on, all requests for client manifests will be associated with the new event, if no specific EventID is specified in the request.:

```
http(s)://<domain>/<path>/<ChannelName>.isml/.mpd
```

To specify an EventID in a request, use the following syntax (where `EventID` should be replaced with the actual EventID and `Manifest` may be replaced to specify any other output format):

```
./<ChannelName>.isml/Events(<EventID>)/.mpd
```

Specifics for Expression Encoder 4

Expression Encoder 4 has a built-in option for using EventIDs. However, using this feature in combination with USP will cause the encoder to not reconnect for a new session. Therefore, it is advised not to use the Expression Encoder's built-in option for EventIDs.

Ingest F4M (deprecated)

Unified Origin also ingests live F4M streams. This is the playlist format used by HTTP Dynamic Streaming. The web server module uses the F4M playlists (and bootstrap and fragments) as its source format and makes the live presentation available in the different supported formats (HSS, HLS, DASH).

Create a server manifest file with the URL to the F4M stream as input:

```
#!/bin/bash

mp4split -o f4m-ingest.isml \
  https://live.unified-streaming.com/smptebitc/smptebitc.isml/.f4m
```

MP4Split fetches the F4M manifest and extracts all the information necessary to create the server manifest file.

By default the DVR window settings are taken from the bootstrap. You can adjust the DVR window by specifying the following server manifest options:

--f4m_dvr_offset_begin

The number of fragments to skip from the beginning of the DVR window. (Defaults to 0)

--f4m_dvr_offset_end

The number of fragments to skip before the end of the DVR window. (Defaults to 0)

For example, say you are using a rolling DVR window and the fragments older than the DVR window are being purged. In that case you may want to set the `f4m_dvr_offset_begin` to an initial value of 2. This make sure that client manifests generated reference only fragments/segments that are still available from the F4M source.

Normally there is no need to adjust the ending of the DVR offset, but some players may be requesting new fragments quite aggressively, while other players may need additional information stored in a fragment about subsequent fragments. The latter is e.g. the case for HTTP Smooth Streaming and you may set the `f4m_dvr_offset_end` to 2 for some additional headroom.

Example command line:

```
#!/bin/bash

mp4split -o f4m-ingest.isml \
  --f4m_dvr_offset_begin=2 \
  --f4m_dvr_offset_end=2 \
  https://live.unified-streaming.com/smptebitc/smptebitc.isml/.f4m
```